

PYTHON PARA QUANT FINANCE: DESDE CERO

Guía de Nivelación Absoluta — Material Oficial Complementario de Quant Club

¿Nunca has programado? Ningún problema. Esta guía está diseñada específicamente para perfiles científicos o financieros que dan sus primeros pasos en código. Olvídate del desarrollo web o de aplicaciones complejas; aquí aprenderás única y exclusivamente la sintaxis y lógica necesarias para manipular datos financieros, simular mercados y calcular el precio de derivados desde el primer día.

1. Variables y Tipos de Datos Financieros

En programación, una **variable** es un contenedor con nombre donde guardamos un dato. Python detecta automáticamente qué tipo de dato almacenamos sin necesidad de declararlo formalmente.

Tipo de Dato	Nombre en Python	Ejemplo en Quant Finance
Número Entero	int	Número de acciones en cartera o días a vencimiento (e.g., 252)
Número Decimal	float	Precio de una opción, volatilidad, tipo de interés (e.g., 0.05)
Texto / Cadena	str	Ticker identificador del activo (e.g., "AAPL" o 'BTC')
Booleano (Verdad/Falso)	bool	Estado de una orden o barrera (e.g., True o False)

```
# Definición de variables en nuestro laboratorio financiero
precio_subyacente = 105.50      # Un número decimal (float)
strike_ejercicio = 100          # Un número entero (int)
ticker = "SAN.MC"               # Texto identificativo (str)
opcion_en_beneficio = True      # Booleano (bool)

# Imprimir los valores en pantalla
print(precio_subyacente)
```

2. Estructuras de Datos Cruciales: Listas y Diccionarios

Para trabajar con series temporales o colecciones de parámetros, necesitamos agrupar nuestras variables individuales.

Listas (Secuencias Ordenadas)

Las listas se delimitan con corchetes `[]` y permiten guardar una secuencia cronológica de precios. En Python, el primer elemento ocupa siempre la **posición 0**.

```
# Histórico de precios semanales de una acción
precios = [98.2, 99.5, 101.0, 100.4, 103.1]

# Acceder a elementos específicos
primer_precio = precios[0] # 98.2
ultimo_precio = precios[-1] # 103.1 (el signo menos empieza desde el final)

# Añadir un nuevo precio de cierre
precios.append(105.2)
```

Diccionarios (Clave - Valor)

Los diccionarios usan llaves {} y sirven para estructurar las propiedades o parámetros de un contrato financiero sin confundir sus valores.

```
# Definición de los parámetros de una opción europea
contrato_opcion = {
    "ticker": "BBVA",
    "tipo": "Call",
    "strike": 6.50,
    "volatilidad": 0.22,
    "vencimiento_anos": 0.5
}

# Acceder a un parámetro concreto de forma intuitiva
strike_actual = contrato_opcion["strike"] # Devuelve 6.50
```

3. Control de Flujo: Condicionales y Bucles

Para que nuestro código tome decisiones autónomas (como en la gestión de riesgos o Delta-Hedging) o repita procesos (como en las simulaciones), usamos condicionales y bucles.

Condicionales (if, elif, else)

Permiten ejecutar bloques de código solo si se cumple una determinada condición matemática.

```
precio_mercado = 104.0
strike = 100.0

if precio_mercado > strike:
    print("La opción está In-the-Money (ITM)")
elif precio_mercado == strike:
    print("La opción está At-the-Money (ATM)")
else:
    print("La opción está Out-of-the-Money (OTM)")
```

Bucles (for)

Permiten recorrer colecciones de datos o repetir una misma tarea un número determinado de veces. ****Nota fundamental:**** En Python, la estructura y pertenencia de un bloque se define mediante el espacio o sangrado (indentación).

```
# Calcular el cambio absoluto entre días secuenciales
precios_historicos = [100, 102, 101, 104]

for p in precios_historicos:
    print(f"Procesando precio de mercado: {p} EUR")
```

4. Funciones: Tu Fábrica de Modelos

Una función es un bloque de código reutilizable que acepta unos parámetros de entrada, realiza un cálculo matemático interno y devuelve un resultado mediante la instrucción `return`. Esto evita reescribir código innecesariamente.

```
# Definición de la función para calcular el Payoff (pago al vencimiento) de una Call
def calcular_payoff_call(precio_terminal, strike):
    # El beneficio es el máximo entre (S_T - K) y cero
    if precio_terminal > strike:
        return precio_terminal - strike
    else:
        return 0.0

# Uso de la función con datos reales
resultado_operacion = calcular_payoff_call(precio_terminal=112.0, strike=100.0)
print(f"El payoff final de la opción es: {resultado_operacion} EUR") # Devuelve 12.0
```

5. El Ecosistema Quant: Librerías Esenciales

Python no viene solo; su gran poder radica en las extensiones especializadas llamadas ****librerías****, que importamos con la palabra clave `import`.

NumPy (Cálculo Numérico y Matrices Vectorizadas)

Los Quants nunca usan bucles tradicionales para simular miles de escenarios porque son lentos. Usamos NumPy para hacer operaciones en bloque (vectorización).

```
import numpy as np

# Crear una lista de precios y transformarla en un array numérico de alto rendimiento
precios_lista = [100.0, 105.0, 110.0]
precios_array = np.array(precios_lista)

# Operación vectorizada directa (multiplica todos los precios a la vez sin usar bucles)
precios_con_shock = precios_array * 1.02 # Aplica un +2% instantáneo
print(precios_con_shock) # [102. , 107.1, 112.2]
```

Pandas (El Excel de los Programadores)

Pandas introduce el objeto DataFrame, que estructura tablas de datos con columnas y filas indexadas por fechas de forma idéntica a una hoja de cálculo.

```
import pandas as pd

# Creación manual de una tabla de datos financieros
datos = {
    "Precio_Cierre": [100, 101.5, 100.8, 102.3],
    "Volumen": [15000, 18200, 12000, 21000]
}
df = pd.DataFrame(datos)

# Calcular métricas estadísticas automáticas de nuestras columnas
print(df["Precio_Cierre"].mean()) # Calcula la media aritmética del precio
```



Consejo de Oro para la Formación: No intentes memorizar toda la sintaxis de golpe. Al principio, programar consiste en replicar los bloques de código que ves en las clases, entender qué parámetros modificas en las entradas y observar cómo impacta en las salidas de tus gráficos o simuladores de Monte Carlo.